

Next-Generation Indexing Structures

Ahmed Mohammady 202301826 Magdy Tarek 202301670

Abstract

Query performance in a database system depends heavily on how data is indexed. A well-chosen index can reduce a table scan from millions of disk reads down to a handful. The B⁺ Tree has long been the default answer to this problem, and it remains dominant in relational engines today. However, workloads have shifted dramatically: write rates are higher, datasets are larger, and a growing class of systems runs entirely in memory. These shifts expose limitations in the B⁺ Tree that were not critical when it was designed.

This report examines three indexing structures that have emerged to address those limitations: the Log-Structured Merge Tree, the Adaptive Radix Tree, and Learned Indexes and critically evaluates where each one fits, where it falls short, and what the overall trend in index design tells us about modern database engineering.

1. Background: Why the B⁺ Tree Is Not Enough

To understand why next-generation structures matter, it helps to be precise about what the B⁺ Tree is optimised for and what it is not.

A B⁺ Tree maintains a sorted, balanced tree where only leaf nodes hold actual record data. Internal nodes carry separator keys that route a lookup downward from root to leaf in $O(\log N)$ steps. Once a leaf is reached, adjacent results can be collected by traversing the sibling pointer chain without climbing back up the tree. This makes range scans cost $O(\log N + K)$ for K results an important property for queries like `WHERE salary BETWEEN 5000 AND 8000`.

The structure excels at read-dominated, mixed OLTP workloads. Its problems emerge under sustained writes. Every insertion must navigate to a specific leaf page, load it from disk if it is not cached, and modify it in place. When a page reaches its capacity limit, it splits sometimes causing a chain of splits up to the root, each one touching and dirtying an additional page. Under high write throughput, this pattern generates significant random I/O and inflates the number of dirty pages the buffer pool must eventually flush to disk.

Space efficiency is also a concern. Pages are kept deliberately under-full typically at 50–70% occupancy to delay the next split. This means roughly a third of allocated storage is unused at any given time.

These are not design flaws; they are consequences of the fundamental choice to keep data sorted and immediately readable at the cost of write-time overhead. The structures below make different choices.

2. Log-Structured Merge Trees

2.1. Core Mechanism

An LSM Tree [1] eliminates in-place writes by treating disk as an append-only medium. Every incoming mutation whether an insert, update, or delete is written first into an in-memory sorted buffer called the MemTable. Deletes are not executed immediately; they are recorded as tombstone markers. When the MemTable reaches a size threshold, it is sealed and flushed to disk as an immutable sorted file called an SSTable. The DBMS accumulates SSTables over time and periodically runs a background merge process called compaction, which consolidates them, resolves duplicate keys by keeping the most recent version, and discards tombstones.

The consequence of this design is that every disk write is sequential. There are no in-place modifications, no random seeks, and no page splits. Write throughput on both HDDs and SSDs improves substantially compared to the B⁺ Tree, which is why LSM-based engines underpin high-ingestion systems like Apache Cassandra, RocksDB, and Google Bigtable.

2.2. The Write Amplification Trade-off

The cost of this architecture is measured by the Write Amplification Factor (WAF):

$$\text{WAF} = \frac{\text{bytes physically written to disk}}{\text{bytes written by the application}}$$

A WAF of 1 means every byte is written to disk exactly once. A WAF of 20 means the same data is rewritten twenty times across compaction rounds. High WAF accelerates SSD wear and consumes I/O bandwidth that would otherwise serve read queries.

The compaction strategy is the primary control over WAF. **Leveled compaction**, the default in RocksDB, organises SSTables into levels where each level is a single non-overlapping sorted run. A file is always merged with all overlapping files in the next level, which limits read amplification, any key exists in at most one SSTable per level but pushes WAF to 10–30×. **Size-tiered compaction**, used in Apache Cassandra, groups files of similar size into tiers and merges a full tier at once. WAF is much lower, but multiple stale versions of the same key can coexist across tiers, increasing the cost of reads. **FIFO compaction** discards old SSTables once a size limit is reached, producing a WAF near 1, but it is only applicable to time-series data with a defined expiry.

Strategy	WAF	Read Amp.	Space Amp.	Suitable For
Leveled	High (10–30×)	Low	Low	Balanced read/write
Size-Tiered	Low	High	High	Bulk write ingestion
FIFO	≈ 1	Medium	Low	Expiring time-series

Table 1: LSM compaction strategy trade-offs.

This is a direct expression of the RUM conjecture: no index can simultaneously minimise Read cost, Update cost, and Memory (space) cost. Each compaction strategy occupies a different point in that three-way trade-off.

3. The Adaptive Radix Tree

3.1. Motivation and Design

When the working dataset fits in main memory, disk I/O is no longer the bottleneck. The constraint shifts to CPU cache efficiency. A B⁺ Tree in memory still requires traversing several large nodes per lookup, each potentially on a different cache line, causing cache misses that stall the CPU pipeline.

The Adaptive Radix Tree (ART) [2] addresses this by using a trie structure that processes keys byte by byte. At each level, the current byte of the search key selects the next child. The tree’s height is therefore bounded by the key length k , not the dataset size N , giving lookup complexity $O(k)$. For a 64-bit integer, k is always 8 bytes regardless of whether the table has one thousand or one billion rows.

What distinguishes ART from a plain trie is its adaptive node layout. Rather than allocating a fixed 256-slot child array at every node, ART uses four node types that grow as needed:

- **Node4** : up to 4 children, stored as a compact key-pointer pair array.
- **Node16** : up to 16 children; key comparison uses SIMD to check all 16 slots in a single instruction.
- **Node48** : up to 48 children via a 256-entry index into a pointer array.
- **Node256** : full direct-mapped 256-pointer array; child access is constant time by byte value.

Most nodes in a real tree are sparse and therefore represented as Node4 or Node16, which fit within one or two CPU cache lines. Lookup traversal causes far fewer cache misses than a B⁺ Tree traversal in memory.

3.2. Limitations

ART is a volatile, memory-resident structure. It produces no durable state on its own, so any system using ART must maintain a separate write-ahead log for crash recovery. When memory pressure forces data to spill to disk, ART has no native overflow path and must be paired with a disk-based layer such as an LSM Tree. Concurrent access is also more complex to implement correctly in ART than in a B⁺ Tree, where latch crabbing is a well-understood protocol.

ART is used as the primary index in HyPer, VoltDB, and SingleStore. It is a strong choice for in-memory OLAP and HTAP systems but is not a standalone solution for disk-based engines.

4. Learned Indexes

4.1. The Core Idea

Kraska et al. [3] proposed reframing index lookup as a regression problem. Given a sorted array of N records, an index is a function $f(\text{key}) \rightarrow \text{position}$. A B⁺ Tree computes this function through recursive key comparisons; a learned index computes it through model

inference. If a machine learning model can approximate f accurately enough, each lookup reduces to a forward pass through that model a few multiply-accumulate operations — followed by a bounded local scan to correct any prediction error.

The Recursive Model Index (RMI) implements this as a two-level cascade. A root model narrows the predicted position to a range, and a leaf model within that range refines the estimate. On static datasets with smooth key distributions, RMI achieves throughput comparable to a tuned B⁺ Tree while using orders of magnitude less memory.

4.2. Practical Constraints

The limitations of learned indexes are significant. A trained model encodes one specific key distribution. Insertions that shift the distribution degrade prediction accuracy, and retraining is expensive enough that it cannot happen inline with write traffic. This makes learned indexes effectively read-only in their basic form.

The ALEX structure [4] partially resolves this by pairing the learned model with a gap array: empty slots are pre-allocated within the sorted layout so that nearby insertions can be absorbed without immediate retraining. ALEX achieves competitive write performance on moderately dynamic workloads but still does not match LSM Trees under high-throughput ingestion.

A more robust deployment pattern is to use the learned component as an advisory layer. The model identifies cases where the conventional index is likely to choose a suboptimal access path, and the system uses that signal to select an alternative strategy. This preserves the correctness guarantees of the traditional structure while harvesting the distribution-awareness of the model.

5. Comparative Analysis

Structure	Point Lookup	Range Scan	Write Throughput	Deployment Context
B ⁺ Tree	$O(\log N)$	$O(\log N + K)$	Moderate	General OLTP
LSM Tree (Leveled)	$O(\log N)$	$O(\log N + K)$	High	Write-heavy, SSD
ART	$O(k)$	$O(k + K)$	High	In-memory OLAP
Learned (RMI/ALEX)	Near $O(1)$	Near $O(1 + K)$	Low	Static analytics

Table 2: Summary comparison of indexing structures.

Two observations emerge from this comparison. First, each structure addresses exactly one failure mode of the B⁺ Tree: LSM Trees target write I/O, ART targets cache latency in memory, and learned indexes target memory footprint on read-only data. None of them is a general replacement.

Second, and more importantly, production systems do not choose one index and apply it everywhere. PostgreSQL supports B⁺ Trees, hash indexes, BRIN indexes, and GiST indexes simultaneously, each serving different access patterns on the same tables. TiDB deploys a B⁺ Tree for transactional row storage alongside an LSM-backed columnar engine for analytics, serving both workloads from a single dataset without duplication. The

dominant architectural pattern is therefore not index replacement but index specialisation: maintain multiple structures, each tuned to a specific query type, and let the query optimiser select the right one at plan time based on cost estimates.

This also means that index selection is shifting from a schema-time decision made by a DBA to a runtime decision made by the cost-based optimiser. As cardinality estimation improves whether through better statistics or learned models, the optimiser can make these selections more reliably, reducing the need for manual index tuning.

6. Conclusion

The B⁺ Tree is not obsolete. It remains the most balanced general-purpose index available, and decades of engineering have made it highly reliable under concurrent access. What has changed is the assumption that one structure can serve all workloads acceptably. High-throughput write pipelines, sub-millisecond in-memory lookups, and large-scale read-only analytics each stress the B⁺ Tree in ways that purpose-built alternatives handle more efficiently.

LSM Trees reframe writes as sequential appends at the cost of compaction overhead. ART achieves cache-efficient in-memory lookup at the cost of requiring external durability mechanisms. Learned indexes reduce memory footprint and lookup latency on stable distributions at the cost of fragility under writes and distribution shifts.

The broader lesson is that index design is fundamentally a trade-off allocation problem. A database engineer who understands where each structure places its costs — across read amplification, write amplification, space amplification, and latency — can compose them into a system that serves the full workload, rather than compromising on all dimensions trying to make one structure do everything.

References

- [1] O’Neil, P., Cheng, E., Gawlick, D., & O’Neil, E. (1996). The log-structured merge-tree (LSM-tree). *Acta Informatica*, 33(4), 351–385.
- [2] Leis, V., Kemper, A., & Neumann, T. (2013). The adaptive radix tree: ARTful indexing for main-memory databases. *Proceedings of IEEE ICDE 2013*.
- [3] Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). The case for learned index structures. *Proceedings of ACM SIGMOD 2018*.
- [4] Ding, J., et al. (2020). ALEX: An updatable adaptive learned index. *Proceedings of ACM SIGMOD 2020*.